Unit -1
Puppet Basic

Unit -1 Syllabus

- Configuration Management System: Configuration Management,
- Pull, Push,
- Introduction to Puppet: Puppet, Why Puppet, Components of Puppet, Puppet Architecture: Puppet Master: Manifest, Template, Files, Certificate Authority, Puppet Clients: Agent, Facter, Installation of Puppet, Puppet Development in Isolation.

Configuration Management System

- Configuration Management (CM) is the practice of handling changes systematically so that a system maintains its integrity over time.
- Think of it like a recipe book for your servers - it ensures that every server is configured exactly the same way, every time.

Real-World Analogy

- Imagine you own a chain of restaurants. You want every restaurant to have:
  - The same menu
  - Same cooking procedures
  - Same staff uniforms
  - Same table arrangements
- Configuration management is like having a detailed manual that ensures consistency across all locations.

- In DevOps, Configuration Management (CM) plays a central role in automating and standardizing environments to enable continuous integration, delivery, and deployment.

- It ensures systems are consistent, scalable, and maintainable, especially across development, testing, staging, and production.

Key Benefits of Configuration Management

- Consistency: All servers have identical configurations
- Scalability: Easy to manage hundreds or thousands of servers
- Reliability: Reduces human errors
- Auditability: Track what changes were made and when
- Disaster Recovery: Quick restoration of systems

Business Impact Example

- Scenario: An e-commerce company with 100 web servers

- Without CM: Manual configuration takes 2 hours per server = 200 hours
- With CM: Automated configuration takes 30 minutes for all servers = 0.5 hours
- Time Saved: 199.5 hours = $19,950 (assuming $100/hour labor cost)

Popular Configuration Management Tools in DeVops



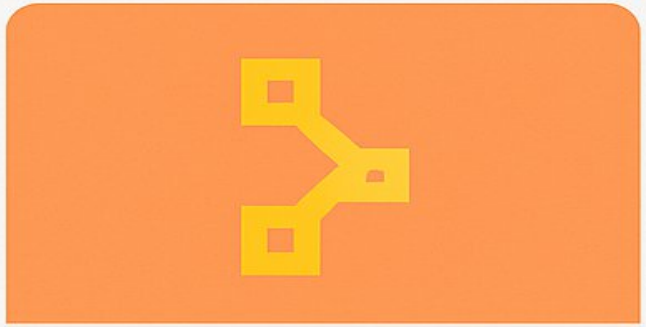
Ansible



Chef



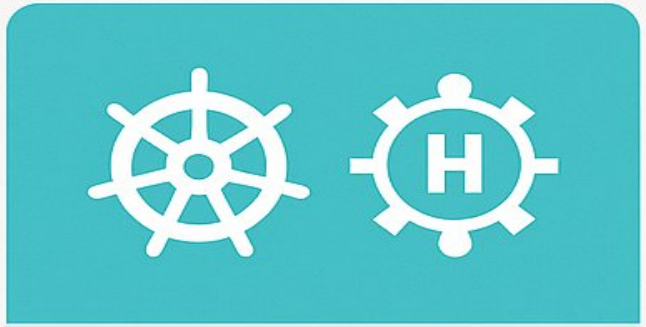
Terraform



Puppet



SaltStack



kubernetes + Helm

Tool	*Description*	*Language*	*Use Case*
Ansible	**Agentless, simple syntax (YAML)**	**YAML**	**Provisioning, app deployment, configuration**
Puppet	**Model-driven, uses declarative DSL**	**Puppet DSL (Ruby-like)**	**Complex infrastructure, enterprise CM**
Chef	**Code-driven, good for complex configs**	**Ruby**	**Infrastructure automation, policy enforcement**
SaltStack	**Fast remote execution and CM**	**YAML + Jinja**	**Orchestration and configuration at scale**
Terraform	**Infrastructure provisioning (not CM strictly)**	**HCL**	**IaC to manage cloud infrastructure**
Kubernetes + Helm	**Manages container configurations**	**YAML**	**Declarative config for container orchestration**

Configuration Management in DevOps Pipelines

- Step-by-step Integration:
- **Define Infrastructure as Code (IaC)**: Use tools like Terraform or CloudFormation to describe infrastructure.
- **Set up Configuration Scripts:** Define desired state using Ansible, Puppet, or Chef.
- **Store in Git:** All scripts and configurations are version-controlled.
- **CI/CD Integration:** Use Jenkins, GitLab CI, or GitHub Actions to apply configurations automatically on deploy.
- **Monitor and Audit:** Tools like Prometheus, ELK Stack, or Datadog help track and audit changes.

Link

- https://www.puppet.com/docs/puppet/6/puppet_overview#puppet_ecosystem

Configuration Models

- Configuration management tools use two primary models to deploy configurations.
  - Push
  - Pull
- These terms describe how configuration changes or updates are applied and distributed across infrastructure.

Push Models

- Central control server pushes configurations to target machines/nodes.
- Control server/CM tool initiates all communications
- Target machines are passive recipients(server/client/nodes)

- Examples: Ansible, SaltStack (can work in push mode)

- Advantages :
  - Immediate control over updates
  - Easier to debug (direct command execution)
  - No agent required (in Ansible)

- Disadvantages:
  - Network connectivity issues can block deployments
  - Less scalable for large infrastructures
  - Requires direct/open access to target systems (e.g., SSH)

Pull Model

- Target machines/nodes periodically pulls its configuration or check the central server for updates
- Target machines initiate the communication
- Central server acts as a configuration repository

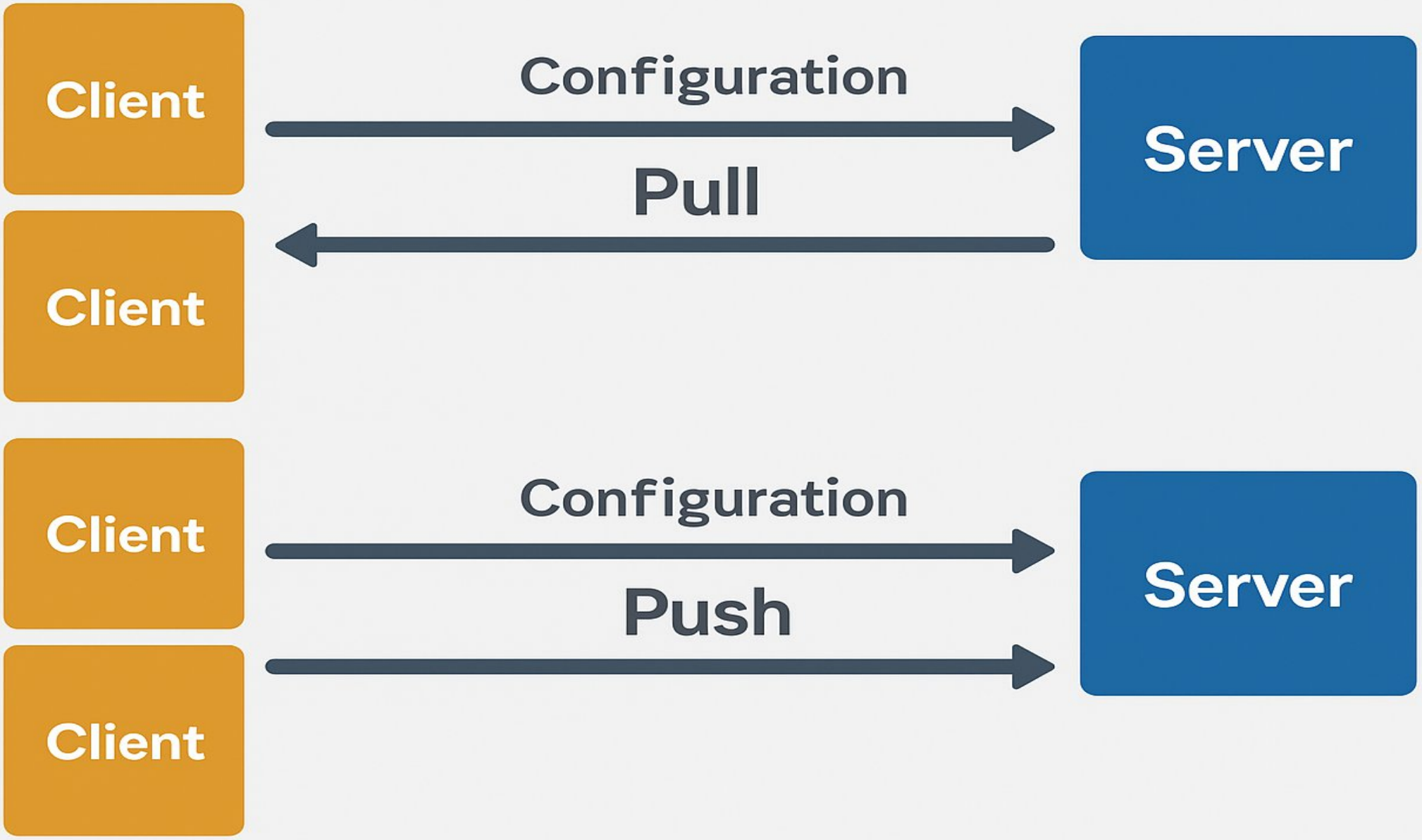
- Examples: Puppet, Chef

- Advantages :
  - More scalable for large environments
  - Nodes self-heal by periodically checking config
  - More secure (central server doesn't need SSH access)
  - More resilient to network issues
  - Targets can work independently
  - Better security (targets initiate connections)

- Disadvantages:
  - Delayed deployment (depends on pull interval)
  - Less immediate control
  - Slower to apply urgent changes
  - Requires agents and proper scheduling

Practical Comparison

Aspect	Push Model	Pull Model
Deployment Speed	Immediate	Delayed (pull interval)
Network Requirements	Outbound from control	Inbound to control
Scalability	Limited	Excellent
Security	Requires access to targets	Targets initiate connections
Offline Tolerance	Poor	Good


Client
Client
Configuration
Pull
Server
Client
Client
Configuration
Push
Server

Example Scenario

- Push: You want to quickly apply a patch to 10 servers — use Ansible to push it instantly.

- Pull: You want 1,000 nodes to stay in a compliant state — use Puppet to have them pull and enforce policies every 30 minutes.

Unit -1
Puppet Basic

Unit -1 Syllabus

- Configuration Management System: Configuration Management,
- Pull, Push,
- Introduction to Puppet: Puppet, Why Puppet, Components of Puppet, Puppet Architecture: Puppet Master: Manifest, Template, Files, Certificate Authority, Puppet Clients: Agent, Facter, Installation of Puppet, Puppet Development in Isolation.

Introduction to Puppet

- Puppet is a open-source configuration management tool that uses a pull-based model to automate the provisioning, configuration, and management of servers and applications across your infrastructure.
- It enables infrastructure as code (IaC) by letting you define desired system states using a declarative language.
- It follows the Infrastructure as Code (IaC) principle, enabling automation and consistency across your servers.
- You can also use Puppet as a deployment tool as it can automatically deploy software on the system.
- Puppet implements infrastructure as code, which means you can test the environment and ensure that it is deployed accurately.

What is Puppet?



Puppet is a configuration management tool

Ensures all systems are configured to the desired states

Puppet is also used as a deployment tool

Automatically deploys software on the system

Puppet implements infrastructure as code

Policies and configurations are written as code

Puppet has two main parts :

Part	Description
Puppet Master (Server)	Central system that stores configuration (manifests)
Puppet Agent (Client)	Installed on every machine you want to configure

Why Choose Puppet?

1. Declarative Language
   - Instead of writing scripts that say
   - "how to do something,"
   - you write manifests that describe
   - "what you want."

```
# Instead of writing a script:
# 1. Check if Apache is installed
# 2. If not, install Apache
# 3. Start Apache service
# 4. Enable Apache to start on boot

# You simply declare what you want:
package { 'apache2':
  ensure => installed,
}

service { 'apache2':
  ensure => running,
  enable => true,
}
```

2. Idempotency:

- Puppet ensures that applying the same configuration multiple times produces the same result without causing issues.

Real-world example:

First run: Installs Apache, starts service

Second run: Checks Apache is installed and running, does nothing

Third run: Still does nothing (system already in desired state.

3. Cross-Platform Support

Works on Linux, Windows, macOS, and various Unix systems.

4. Large Community and Enterprise Support

- Open-source version available
- Enterprise version with additional features
- Large community contributions
- Extensive module library (Puppet Forge)

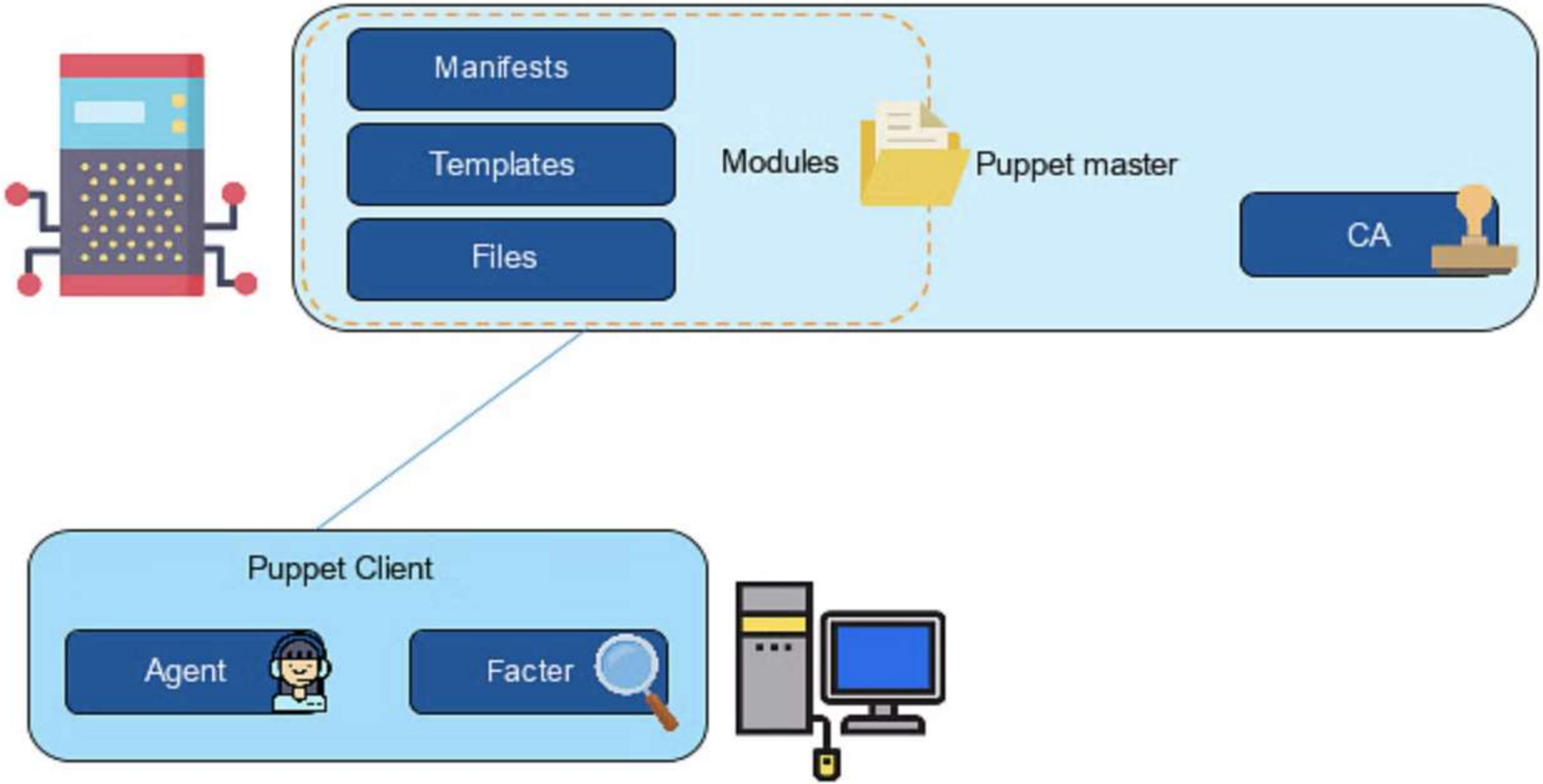
Key Use Cases

1. Server Configuration Management
   - Web server configurations
   - Database server setups
   - Application deployments
2. Compliance Management
   - Security hardening
   - Regulatory compliance (SOX, PCI-DSS, HIPAA)
   - Audit trail maintenance
3. Infrastructure as Code
   - Version-controlled infrastructure
   - Reproducible environments
   - Environment promotion (dev → test → prod)

Components of Puppet

- Puppet follows a client-server architecture (with agent/master model), but also supports standalone mode (puppet apply).
- Its components can be grouped into core, data, and execution layers.
- The various components of puppet are :
  - Puppet Master (Server)
  - Puppet Agent (Client)
  - Manifests
  - Modules
  - Facter
  - Catalog

Components of Puppet



- The Puppet environment can be broken down into the main server environment and the client environment.
- In the main server environment, there is a Puppet master store which stores all configuration files.
  - Manifests are the actual codes for configuring the clients
  - Templates combine code and data to render a final document
  - Files are the static content that can be downloaded by the clients
  - Modules are a collection of manifests, templates, and files
  - Certificate authority allows the master to sign the certificates sent by the client
- The Puppet client is the machine that needs to be configured, consisting of Agent and Factor. The agent continuously interacts with the master server to ensure that the certificates are being updated appropriately. The factor collects the current state of the client that is used and communicates it back through the agent.

Puppet Master :

- The central server that stores and serves configurations to client machines.
- Key Functions:
  - Stores manifests and modules
  - Compiles catalogs for agents
  - Manages certificates
  - Serves files and templates

Puppet Agent (Client)

- Software that runs on managed nodes and applies configurations.

Key Functions:

- Requests catalogs from Puppet Master
- Applies configurations to the local system
- Reports back to Puppet Master
- Gathers system facts

3. Manifests
   - Text files written in Puppet's domain-specific language (DSL) that describe the desired state of system resources.

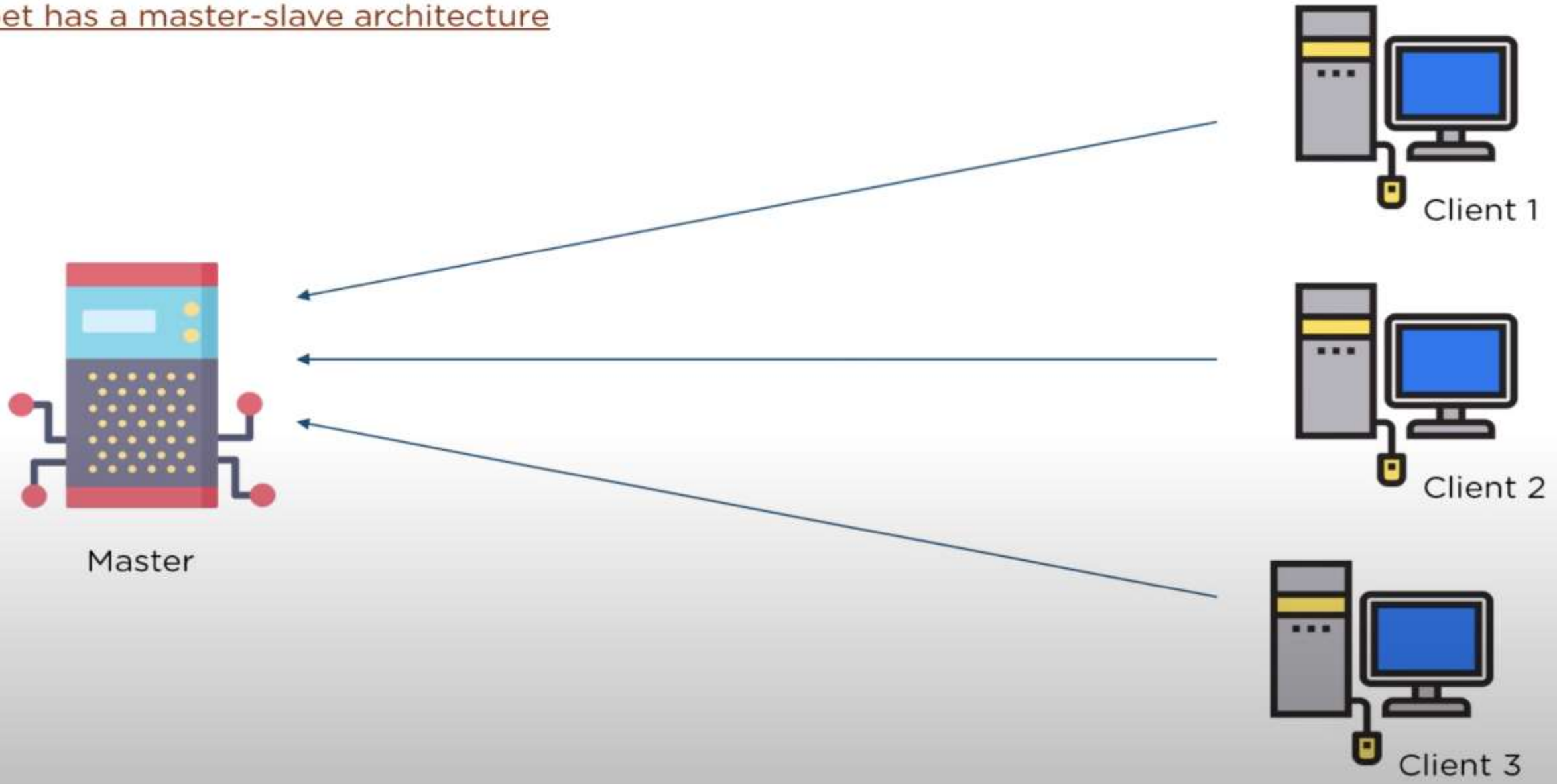
4. Modules
   - Reusable collections of manifests, files, templates, and other resources organized in a specific directory structure.

5. Facter
   - A system inventory tool that gathers facts about nodes (hardware, OS, network configuration, etc.).

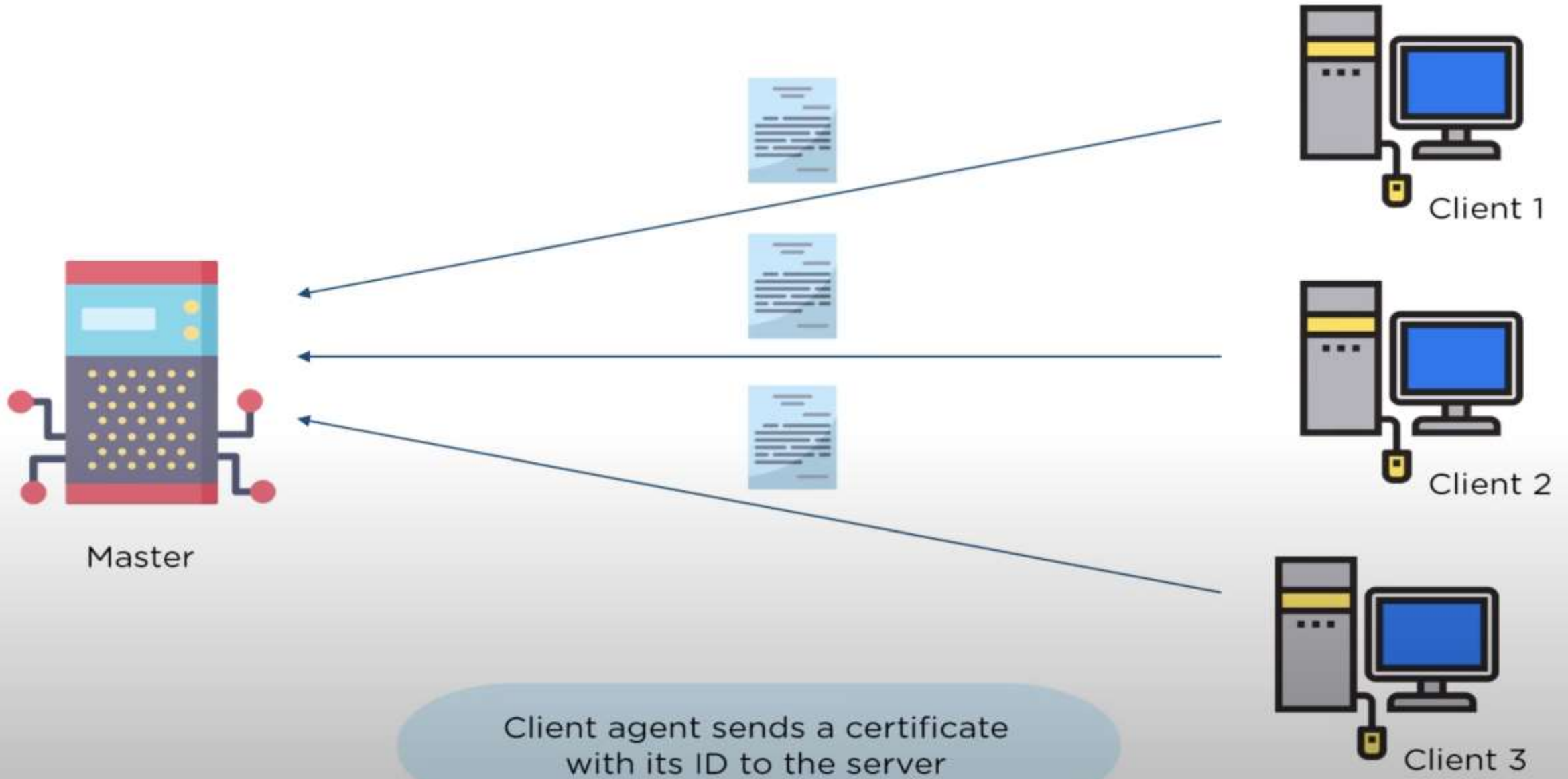
6. Catalog
   - A compiled document that describes the desired state of a particular node, created by the Puppet Master from manifests and sent to agents.

Working of Puppet

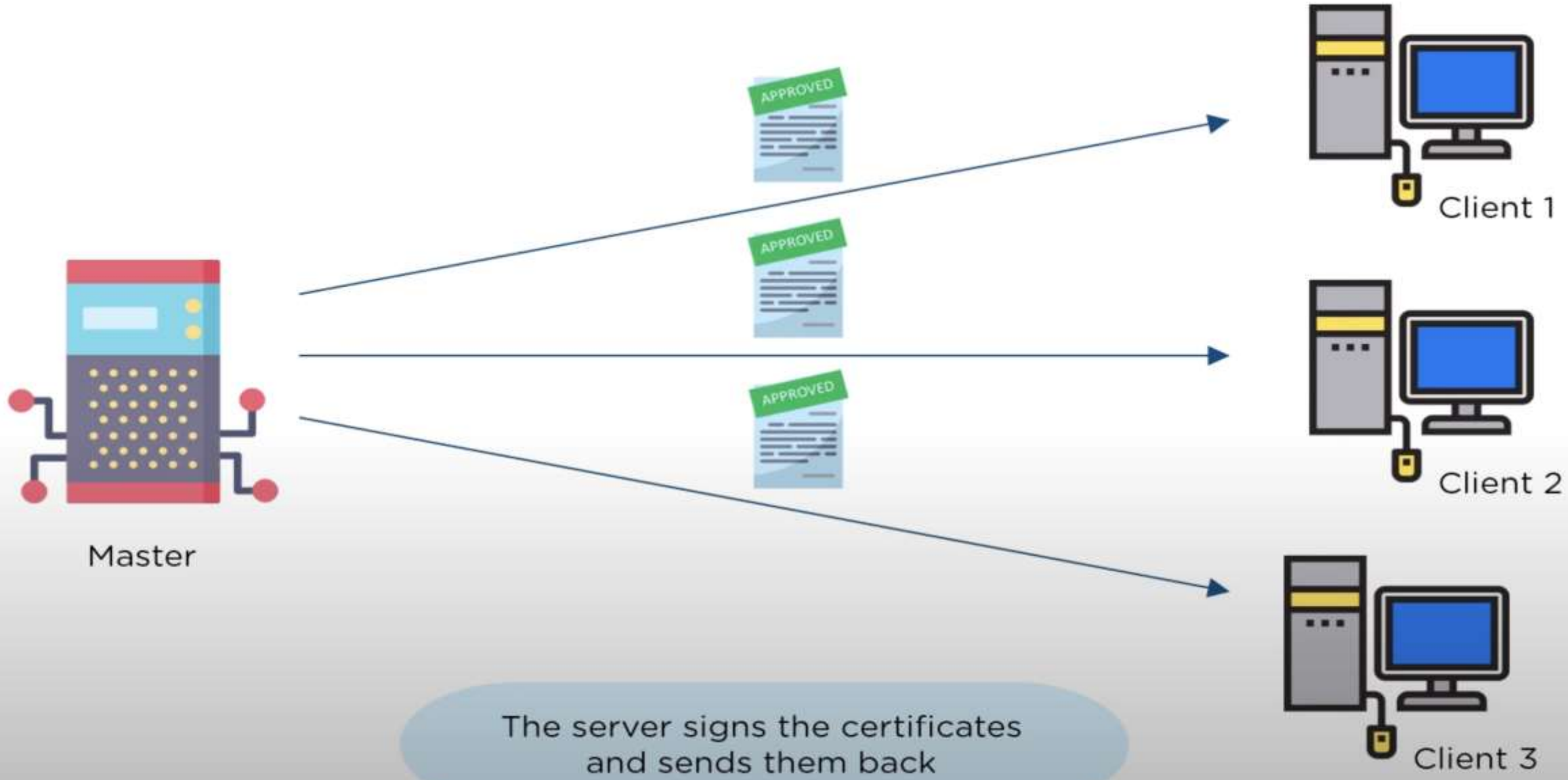
Puppet has a master-slave architecture



Working of Puppet

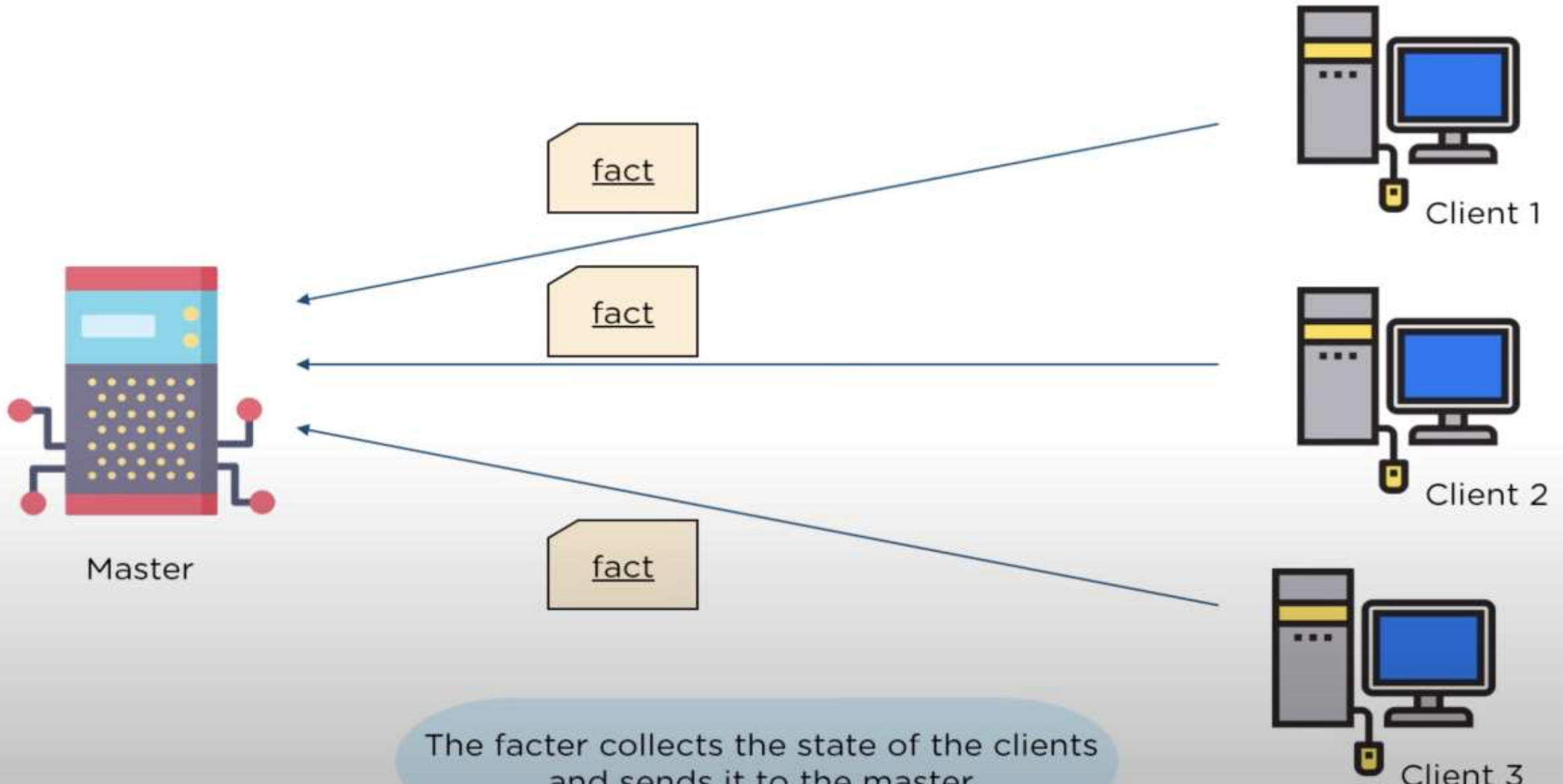


Working of Puppet

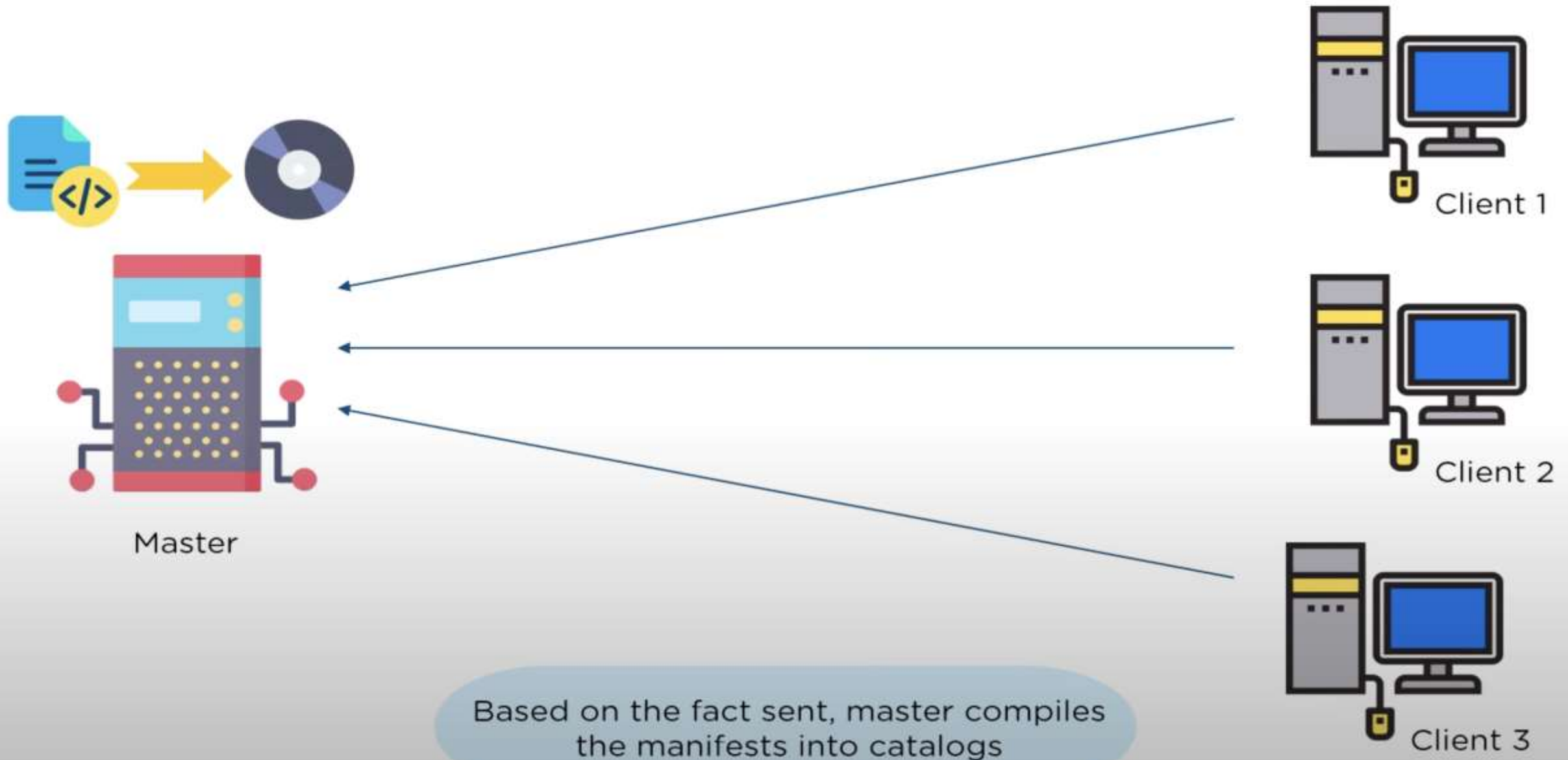


©Simplilearn. All rights reserved.

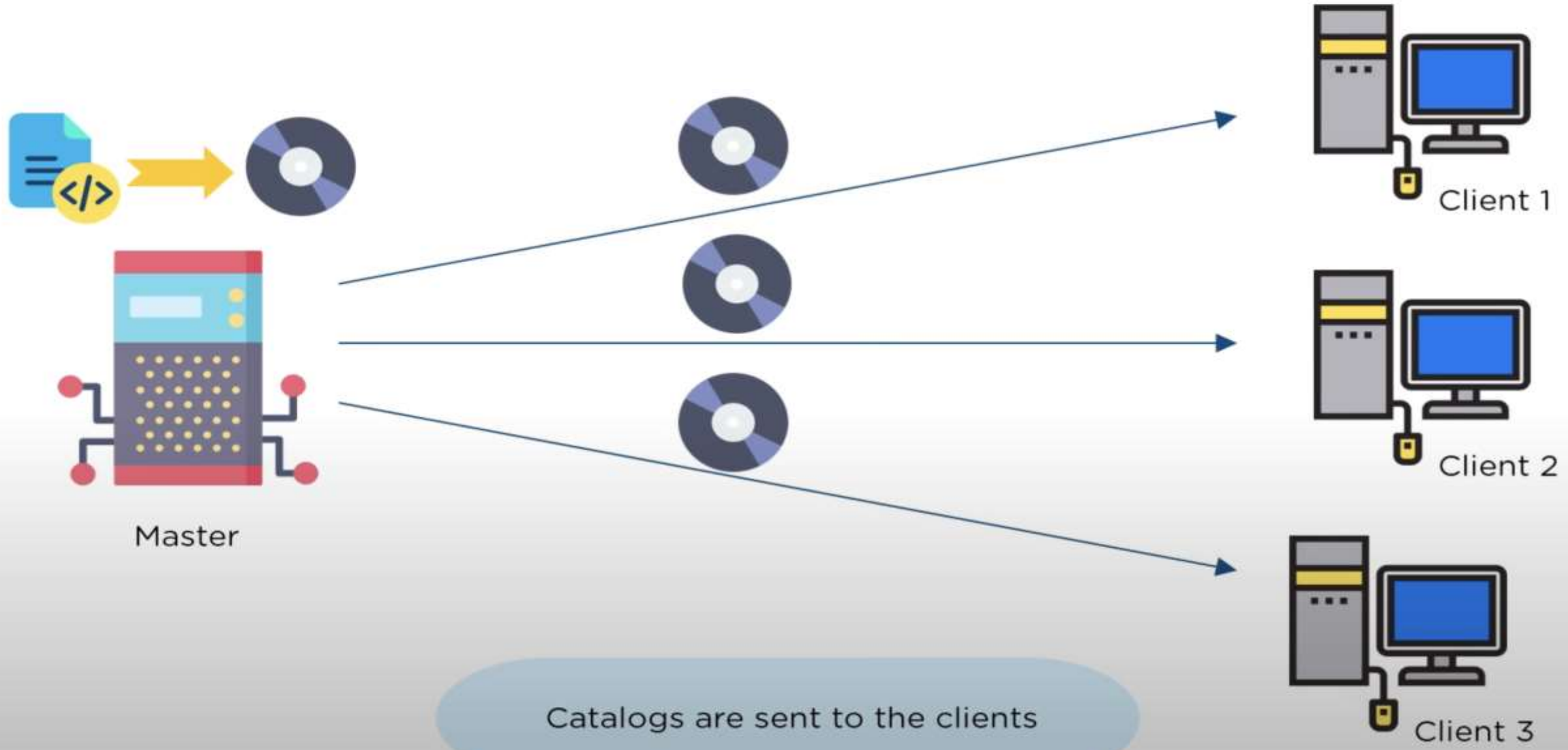
Working of Puppet



Working of Puppet



Working of Puppet

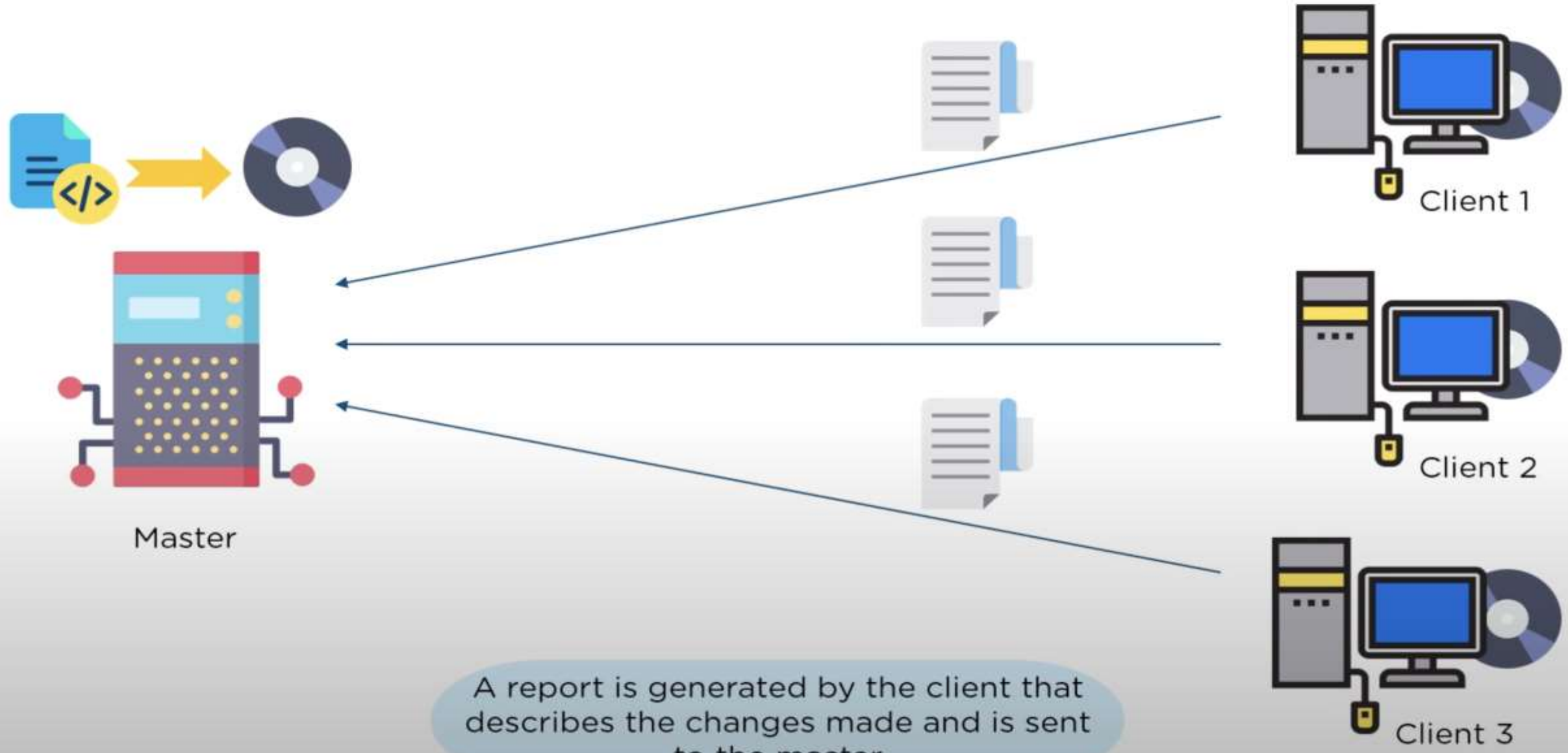


pplilearn. All rights reserved

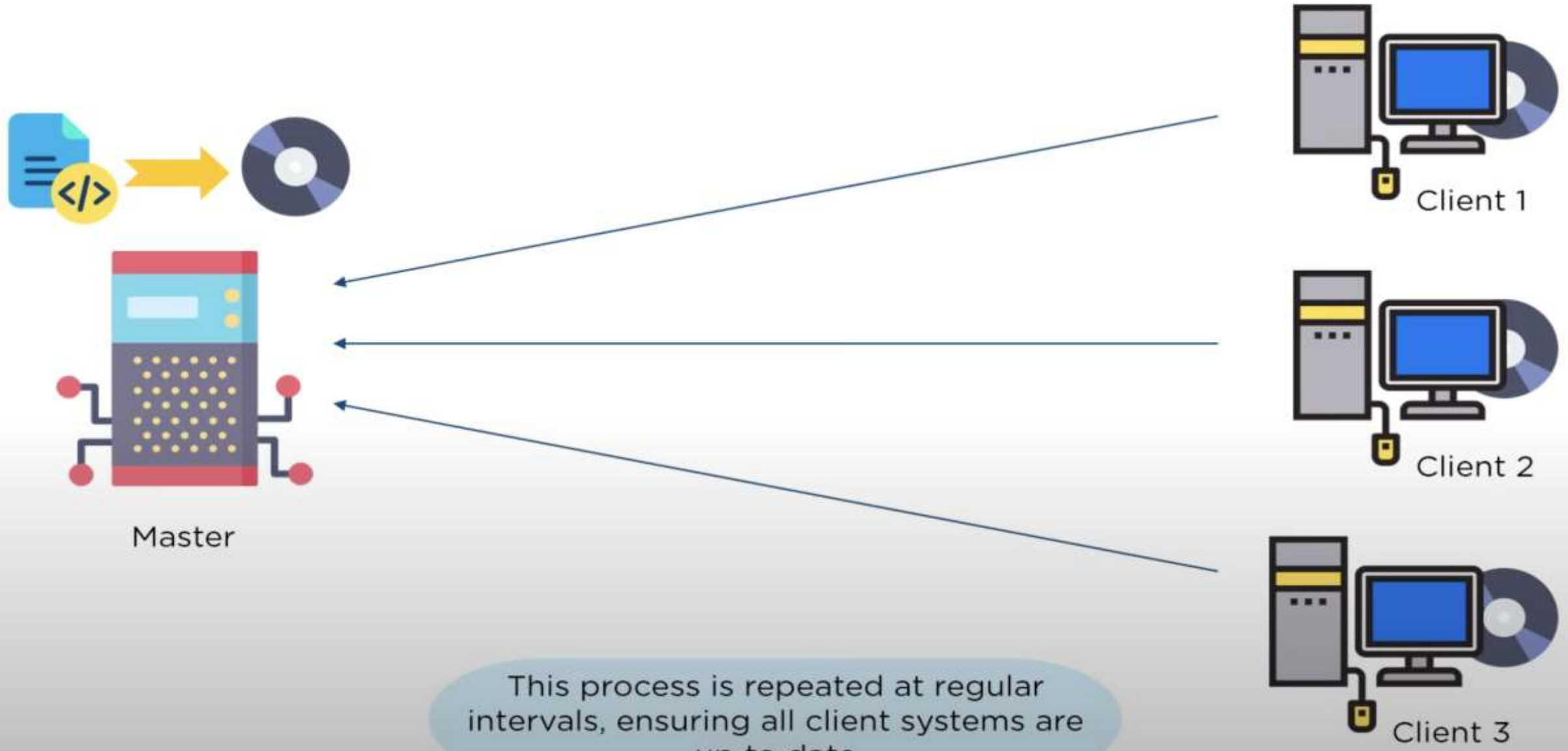
Working of Puppet



Working of Puppet



Working of Puppet

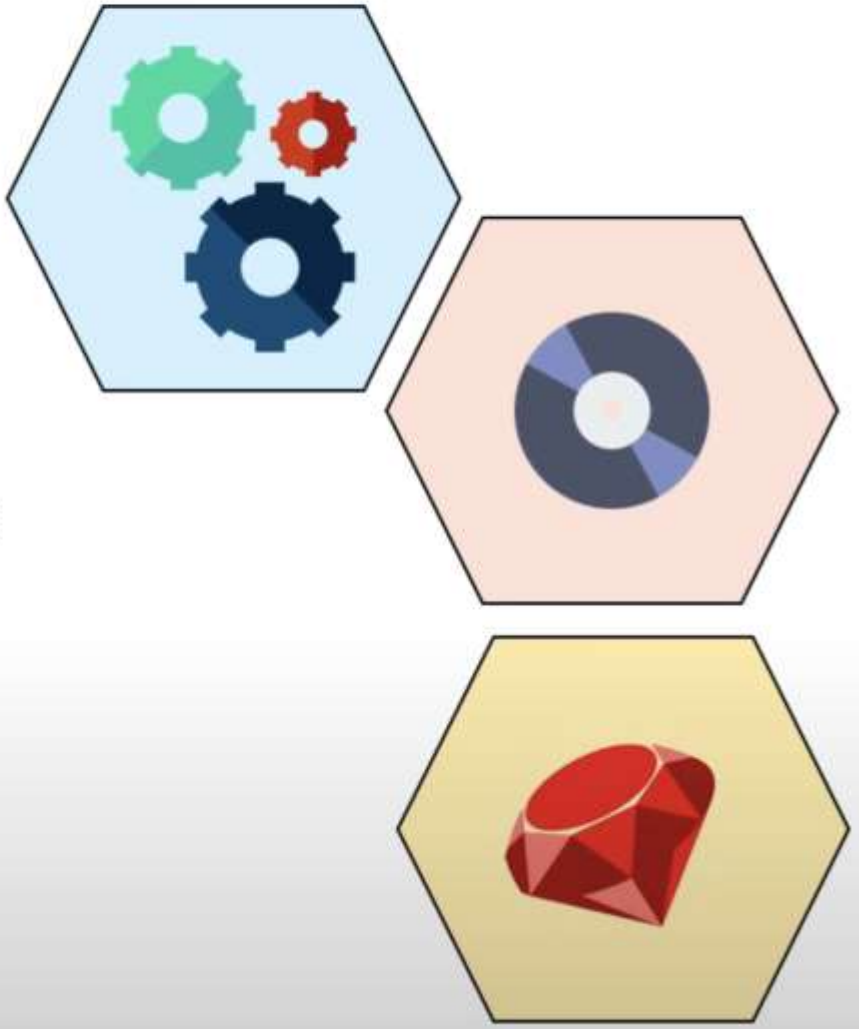


Writing manifests in Puppet

Files written for configuring a node are called manifests

They are compiled into catalogs which are executed at the client

They are written in Ruby with a .pp extension



Writing manifests in Puppet



01 Create

Manifests are written by the system administrator

02 Compile

Manifests are compiled into catalogs

03 Deploy

Catalogs are deployed on the clients

04 Execute

Catalogs are run on the client by the agent

06 End

Clients are configured to desired state

Unit -1
Puppet Basic

Unit -1 Syllabus

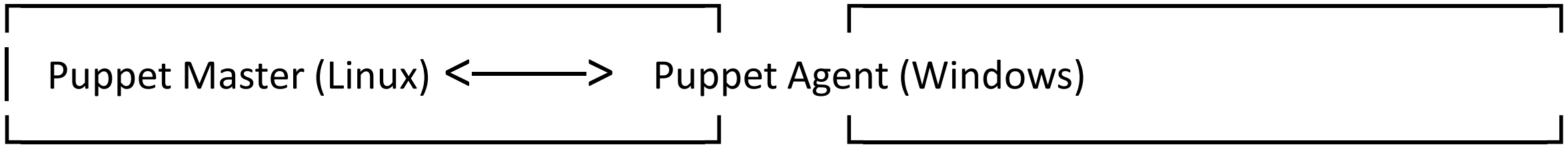
- Configuration Management System: Configuration Management,
- Pull, Push,
- Introduction to Puppet: Puppet, Why Puppet, Components of Puppet, Puppet Architecture: Puppet Master: Manifest, Template, Files, Certificate Authority, Puppet Clients: Agent, Facter, Installation of Puppet, Puppet Development in Isolation.

Puppet Workflow

- Agent requests catalog from Puppet Master
- Master compiles catalog based on manifests and node classification
- Master sends catalog to agent
- Agent applies catalog to local system
- Agent sends report back to master

- Link :
  - https://www.puppet.com/docs/puppet/5.5/puppet_index

Puppet Agent Configuration on Windows – Troubleshooting Connection to Puppet Master



Port 8140 open

Communicates via SSL

Install Puppet Agent on Windows

Step 1: Download the Installer

Download the Puppet Agent for Windows from the official site:
https://downloads.puppet.com/windows/puppet-agent-x64-latest.msi

Step 2: Install Puppet Agent

Run the .msi installer.

During installation, choose default options.

Once installed, open Command Prompt (Admin) and verify:

```
puppet --version
```

Important Puppet Paths on Windows

Item	Path
Puppet conf	C:\ProgramData\PuppetLabs\puppet\etc\puppet.conf
SSL certs	C:\ProgramData\PuppetLabs\puppet\ssl\
Executables	C:\Program Files\Puppet Labs\Puppet\bin\

Check Puppet Config Path Dynamically

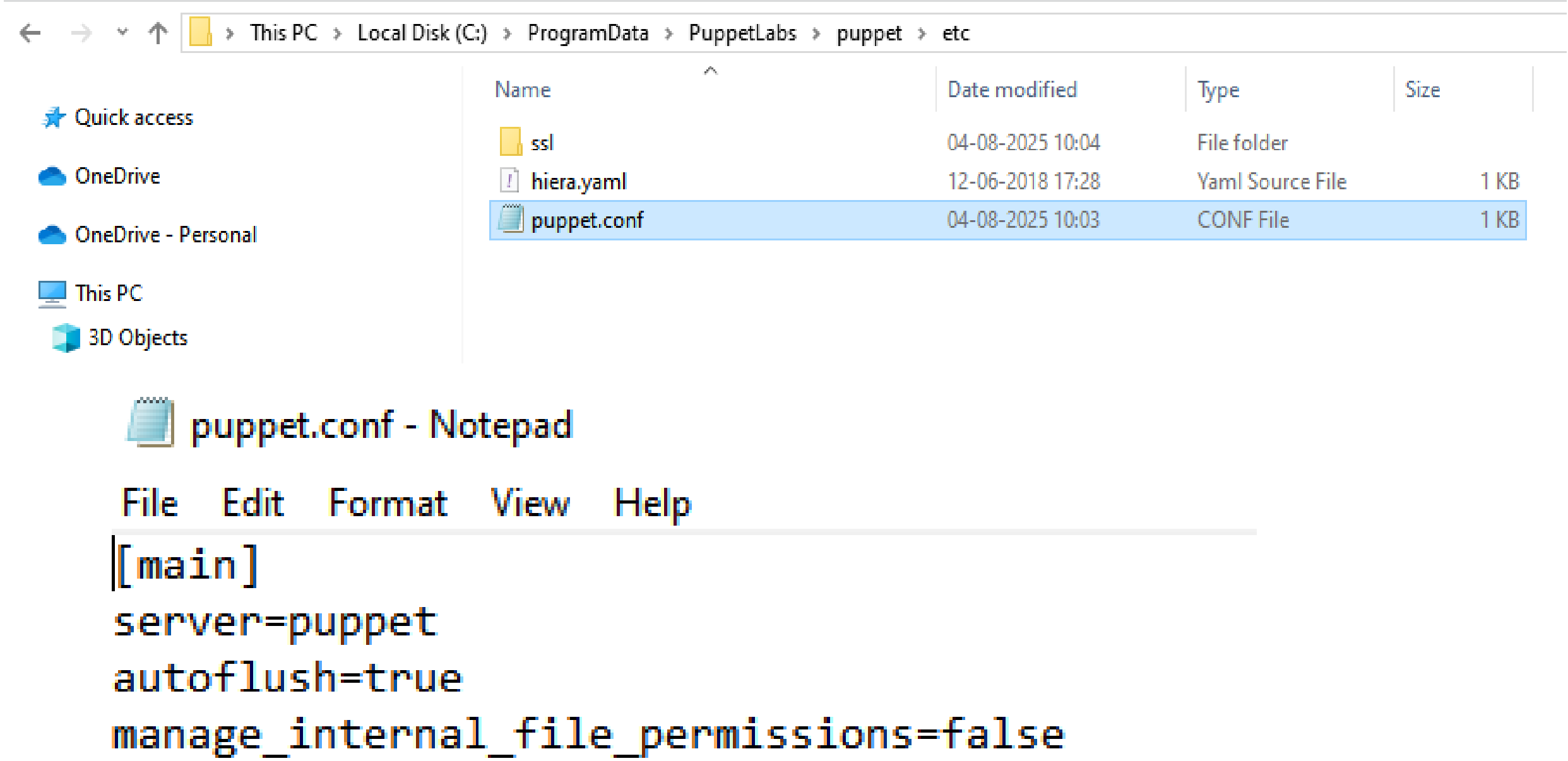
Run this to get where Puppet expects the config directory:

puppet config print confdir

```
C:\Windows\system32>puppet --version
4.10.12

C:\Windows\system32>puppet config print confdir
C:/ProgramData/PuppetLabs/puppet/etc

C:\Windows\system32>
```


This PC › Local Disk (C:) › ProgramData › PuppetLabs › puppet › etc
Quick access
OneDrive
OneDrive - Personal
This PC
3D Objects
Name
Date modified
Type
Size
ssl
04-08-2025 10:04
File folder
hiera.yaml
12-06-2018 17:28
Yaml Source File
1 KB
puppet.conf
04-08-2025 10:03
CONF File
1 KB
puppet.conf - Notepad
File Edit Format View Help
[main]
server=puppet
autoflush=true
manage_internal_file_permissions=false

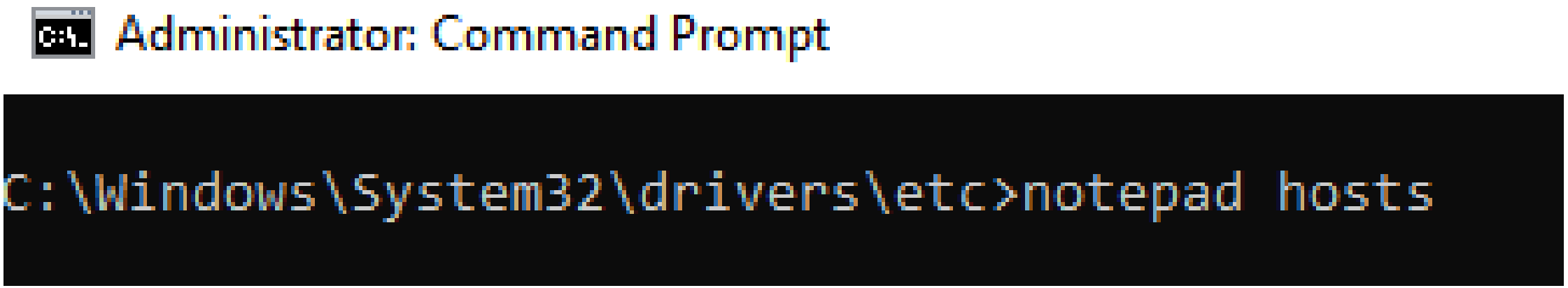
Directive	Description	Real-World Analogy
server=puppet	Specifies the hostname of the Puppet master server (e.g., puppet, puppet.example.com). The agent will communicate with this master.	📞 “Call this number (server) for work instructions.”
autoflush=true	Forces Puppet to immediately flush logs and reports to disk instead of buffering them. Useful for real-time troubleshooting.	💧 “Don't store logs in memory—write them instantly.”
manage_internal_file_permissions=false	Prevents Puppet from enforcing file permissions on internal files (like SSL certs). Useful on Windows to avoid file access issues.	🔒 “Don’t change security settings—we’ll handle it ourselves.”

Real-World Analogy

- **server=puppet**: “This is the boss’s phone number I call for work.”

- **autoflush=true**: “I write down what I’m doing instantly — no waiting.”

- **manage_internal_file_permissions=false**: “I don’t touch file locks — the OS handles it.”


Administrator: Command Prompt
C:\Windows\System32\drivers\etc>notepad hosts

```
C:\Windows\system32>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . : lpu.com

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::f0f8:a644:c1f9:6b44%7
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :
```

hosts - Notepad

File Edit Format View Help

```
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#      102.54.94.97     rhino.acme.com          # source server
#       38.25.63.10     x.acme.com              # x client host

# localhost name resolution is handled within DNS itself.
#       127.0.0.1       localhost
#       ::1             localhost
# Added by Docker Desktop
172.26.8.228 host.docker.internal
172.26.8.228 gateway.docker.internal
# To allow the same kube context to work on the host and the container:
127.0.0.1 kubernetes.docker.internal
192.168.56.1  puppet
# End of section
```

```
C:\Windows\system32>ping puppet

Pinging puppet [192.168.56.1] with 32 bytes of data:
Reply from 192.168.56.1: bytes=32 time<1ms TTL=128
Reply from 192.168.56.1: bytes=32 time<1ms TTL=128
Reply from 192.168.56.1: bytes=32 time<1ms TTL=128
Reply from 192.168.56.1: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.56.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Windows\system32>
```

Important Note

Puppet Server (Master) is not supported natively on **Windows as a master** — it's officially designed to run on Linux systems only (e.g., Ubuntu, CentOS, RHEL).
Windows is supported only for Puppet Agent — not the Master.
So that's likely why nothing is listening on port 8140 — the Puppet Server isn't installed or isn't capable of running on Windows.

- Installing Puppet Agent on Windows
- Configuring puppet.conf and hosts File
- Opening Firewall Port 8140 for Puppet Communication
- Troubleshooting: Puppet Agent Cannot Connect to Master
- Why Puppet Master Doesn’t Work on Windows
- Recommended Setup: Puppet Master on Linux + Windows Agent

macOS Installation

- # Using Homebrew
  - brew tap puppetlabs/puppet
  - brew install --cask puppetlabs/puppet/puppet-agent

- After installation, ensure the path is set:
  - export PATH=/opt/puppetlabs/bin:$PATH
- Verify :
  - puppet --version

```
Last login: Tue Aug  5 10:05:10 on ttys000
divyathakur@Divyas-MacBook-Air ~ % uname -m
arm64
divyathakur@Divyas-MacBook-Air ~ % 15.5
zsh: command not found: 15.5
divyathakur@Divyas-MacBook-Air ~ % brew tap puppetlabs/puppet
==> Auto-updating Homebrew...
Adjust how often this is run with HOMEBREW_AUTO_UPDATE_SECS or disable with
HOMEBREW_NO_AUTO_UPDATE. Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man brew`).
==> Auto-updated Homebrew!
Updated 1 tap (homebrew/cask).
==> New Casks
advantagescope: FRC log analysis tool
codebolt: AI Powered Code Editor

You have 10 outdated formulae and 1 outdated cask installed.

==> Tapping puppetlabs/puppet
Cloning into '/opt/homebrew/Library/Taps/puppetlabs/homebrew-puppet'...
remote: Enumerating objects: 2396, done.
remote: Counting objects: 100% (316/316), done.
remote: Compressing objects: 100% (154/154), done.
remote: Total 2396 (delta 176), reused 171 (delta 158), pack-reused 2080 (from 2)
Receiving objects: 100% (2396/2396), 1.42 MiB | 2.90 MiB/s, done.
Resolving deltas: 100% (1361/1361), done.
Tapped 13 casks and 5 formulae (48 files, 1.6MB).
divyathakur@Divyas-MacBook-Air ~ % brew install --cask puppetlabs/puppet/puppet-agent
==> Downloading https://downloads.puppet.com/mac/puppet/14/arm64/puppet-agent-7.
######################################################################## 100.0%
==> Installing Cask puppet-agent
==> Running installer for puppet-agent with sudo; the password may be necessary.
Password:
installer: Package name is puppet-agent
installer: Installing at base path /
installer: The install was successful.
🍺  puppet-agent was successfully installed!
==> No outdated dependents to upgrade!
divyathakur@Divyas-MacBook-Air ~ % export PATH=/opt/puppetlabs/bin:$PATH

divyathakur@Divyas-MacBook-Air ~ % puppet --version
7.34.0
divyathakur@Divyas-MacBook-Air ~ %
```

- Puppet agent has been successfully installed in windows and mac .
- Now to complete your setup, you'll need to install a Puppet Master (Puppet Server).
- The Puppet Master handles:
  - Managing configurations (manifests, modules)
  - Responding to agent requests over port 8140
  - Certificate authority (CA) for agent SSL certificates

- **The Puppet Master is only supported on Linux — NOT on macOS or Windows.**

Recommended Platforms for Puppet Server:

- Ubuntu 20.04 / 22.04
- Debian 10 / 11
- RHEL / CentOS 7 / 8 / 9
- Rocky / AlmaLinux (RHEL-compatible)

Puppet-master

- install a virtual machine , and create a new virtual machine of ubuntu in which we will use puppet-master .
- iso file for windows :
  - https://ubuntu.com/download/desktop

INFRASTRUCTURE OVERVIEW

Component	OS/Tool	Role
Ubuntu (ARM64)	UTM on macOS	Puppet Master
Ubuntu (x86_64)	VirtualBox on Windows	Puppet Master
macOS	Host OS	Puppet Agent
Windows	Host OS	Puppet Agent

- Run this to get where Puppet expects the config directory(LOCAL):
  - puppet config print confdir
- on ubuntu (VM):
  - sudo apt update
  - sudo apt upgrade -y

To ssh in terminal

- In VM :
  - sudo apt update : to change password
  - ls /home : check users
  - sudo apt install openssh-server
  - sudo systemctl enable ssh
  - sudo systemctl start ssh
  - sudo systemctl status ssh
- On MacOS:
  - ssh username@ipaddress
  - change password : sudo passwd user or root

Installation of puppet master

- # 1. Remove old Puppet completely
  - sudo systemctl stop puppetserver
  - sudo apt purge -y puppetserver puppet-agent puppet
  - sudo rm -rf /etc/puppetlabs /opt/puppetlabs /var/log/puppetlabs /var/opt/puppetlabs
  - sudo apt autoremove -y
  - sudo apt clean
- # 2. Install correct Java
  - sudo apt install -y openjdk-17-jdk
  - sudo update-alternatives --config java
- # 3. Add Puppet 8 repository for Noble
  - wget https://apt.puppet.com/puppet8-release-$(lsb_release -cs).deb
  - sudo dpkg -i puppet8-release-$(lsb_release -cs).deb
  - sudo apt update
- # 4. Install Puppet Server
  - sudo apt install -y puppetserver

- # 5. Lower heap usage just to be safe
  - sudo sed -i 's/Xms2g/Xms512m/; s/Xmx2g/Xmx512m/' /etc/default/puppetserver

- # 6. Set hostname
  - sudo hostnamectl set-hostname puppetmaster.example.com
  - echo "127.0.0.1 puppetmaster.example.com puppetmaster" | sudo tee -a /etc/hosts

- # 7. Start Puppet Server
  - sudo systemctl daemon-reload
  - sudo systemctl enable puppetserver
  - sudo systemctl start puppetserver
  - sudo systemctl status puppetserver

Unit-2
Advanced Puppet

Syllabus

- Puppet Configuration, Managing Packages in Puppet, Puppet Modules,
- Monitoring Web Servers, Load Balancing Clusters, Scaling Puppet Environment, Connecting Puppet
- Agent with Puppet Master, Making Configuration Dynamic, Extending Puppet, Puppet Classes and
- Functions (including Custom Functions), Using Puppet via Command Line, Managing Resources with
- puppet apply Command, Puppet Manifests.

Puppet Configuration

- Prerequisites:
- Puppet installation.
- Puppet Master status:
  - sudo systemctl status puppetserver
- Puppet Agent version:
  - puppet --version

Puppet configuration

- It refers to the setup and management of system resources and software using Puppet, which is a popular configuration management tool.
- Puppet helps automate the process of configuring servers, deploying applications, and enforcing system policies across multiple machines consistently.
- Puppet configuration is the act of defining and applying the desired state of systems using Puppet manifests, modules, and settings, ensuring consistency and automation across many servers.

What Puppet Does

- Puppet allows you to define the desired state of your systems. For example:
  - Which packages should be installed (nginx, mysql, etc.)
  - Which services should be running (apache2, docker)
  - Which files should exist and their contents (/etc/hosts, /etc/myapp/config.yml)
  - User accounts and permissions
- Once defined, Puppet ensures that your systems converge to this desired state automatically.

Key Components of Puppet Configuration

- Puppet configuration involves several components:

Manifests

- Files written in Puppet's domain-specific language (DSL), typically with a .pp extension.
- They describe the desired state of resources.

Modules

- Collections of manifests, files, templates, and other resources.
- Allow for reusability and organization of configurations.
- Example: a mysql module might include installation, configuration, and service management.

- Nodes
  - Define which configuration applies to which system (server).
  - Node definitions allow you to specify different configurations for different servers.
- Facter
  - Puppet uses Facter, a tool that gathers system information (like OS, IP, memory) to help make decisions in manifests.
- Puppet Master and Agents (Classic Architecture)
  - Master: Server that stores all configurations and modules.
  - Agent: Client machine that applies configurations sent by the master.
  - Puppet can also run in a masterless mode, where the manifests are applied directly on the nodes.

How Puppet Configuration Works

- Admin writes manifests/modules defining desired state.
- Puppet agent checks the current system state.
- Puppet compares the current state with the desired state.
- Puppet applies changes to bring the system into compliance.

Puppet Configuration Files

- Some common Puppet configuration files include:
  - puppet.conf → Main configuration file for Puppet (locations, logging, master settings).
  - hiera.yaml → For hierarchical data lookup (variables and secrets).
  - auth.conf → authentication & authorization.
  - Manifest files (*.pp) → Define the desired state.
  - Module directories → Organize reusable code and resources.

- Puppet’s main configuration file (puppet.conf) defines how the master and agents behave.
- It is divided into sections:
  - [main] → Global settings for all Puppet operations.
  - [agent] → Agent-specific settings.
- [master] → Master-specific settings.
- Edit Puppet Master/agent config:
  - sudo nano /etc/puppetlabs/puppet/puppet.conf

Configure puppet.conf

- certname: unique name of node.
- server: Puppet master address.
- runinterval: how often agent checks in.

Managing Packages in Puppet

- In Puppet, managing packages is a core part of configuration management.
- Puppet allows you to install, remove, or upgrade software packages on your nodes in a declarative way.
- Declarative syntax ensures idempotency — running the same manifest twice won’t break the system.
- Puppet has built-in resource types like package, service, and file.
- You typically manage packages in:
- Manifests
  - Puppet code files (.pp) usually located in:
    - /etc/puppetlabs/code/environments/production/manifests/

Puppet uses the package resource type.

- Syntax:

```
package { 'package_name':
ensure => 'installed',          # or 'latest', 'absent', 'purged'
provider => 'apt',              # optional, e.g., apt, yum, dnf, gem
}
```

- Example:
  - Install a package:

```
package { 'vim':
ensure => installed,
}
```

- Install Multiple Packages

```
package { ['git', 'curl', 'wget']:
  ensure => installed,
}
```

- Example:

```
package { 'nginx':
  ensure => installed,
}
service { 'nginx':
  ensure => running,
  enable => true,
  require => Package['nginx'],
}
```

- Run with :
  - puppet apply nginx.pp

Unit-2
Advanced Puppet

Syllabus

- Puppet Configuration, Managing Packages in Puppet, Puppet Modules,
- Monitoring Web Servers, Load Balancing Clusters, Scaling Puppet Environment, Connecting Puppet
- Agent with Puppet Master, Making Configuration Dynamic, Extending Puppet, Puppet Classes and
- Functions (including Custom Functions), Using Puppet via Command Line,
- Managing Resources with puppet apply Command, Puppet Manifests.

Puppet Modules

- A Puppet module is a self-contained unit of Puppet code that includes manifests, files, templates, and other resources to manage a particular system configuration or application.

Modules help you organize and reuse Puppet code by enabling you to split the code into several manifests.

- Modules = reusable bundles of Puppet code. They group manifests, files, and templates.

Why use modules?

- Reusability: Write once, use on multiple nodes.
- Organization: Keep related configurations together.
- Sharing: Puppet Forge allows you to use community modules.

Example use:

A module called apache can install Apache, start the service, and deploy a website file.

Puppet Module Structure

- A typical Puppet module has this structure:

```
my_module/
├── manifests/
│   └── init.pp          # Main class
├── files/               # Static files (config files, scripts)
├── templates/           # ERB templates for dynamic configuration
├── examples/            # Example manifests
├── tests/               # Test manifests
└── metadata.json        # Module metadata
```

Explanation:
*my_module: module root folder
•manifests/: Contains Puppet code (.pp). init.pp defines the main class.
•files/: Static files you want to copy to nodes.
•templates/: ERB templates for dynamic content.
•metadata.json: Describes the module (name, author, dependencies).

Puppet 8 uses the code directory:

```
/etc/puppetlabs/code/environments/production/
├── manifests/          # Your main manifests (.pp files)
│   └── site.pp         # Main entry point
└── modules/            # Your Puppet modules
    └── <module_name>/
        ├── manifests/
        │   └── init.pp
        ├── files/
        └── templates/
```

- Important: On Ubuntu, if these directories don’t exist, you can create them:
  - sudo mkdir -p /etc/puppetlabs/code/environments/production/manifests
  - sudo mkdir -p /etc/puppetlabs/code/environments/production/modules

- Verify Puppet directories:

- Puppet 8 uses this path for environments:
  - /etc/puppetlabs/code/environments/production
- Check if it exists:
  - ls -l /etc/puppetlabs/code/environments
- If environments or production doesn’t exist, create it:
  - sudo mkdir -p /etc/puppetlabs/code/environments/production

Creating a Module in Puppet 8

- Generate a module folder and sub folders :(apache module skeleton):
  - sudo mkdir -p /etc/puppetlabs/code/environments/production/modules/apache/{manifests,files,templates}
  - cd /etc/puppetlabs/code/environments/production/modules/apache
  - ls

  - if Permissions issue:
    - sudo chown -R $USER:$USER /etc/puppetlabs/code/environments/production/modules
- Generate a .pp file :
- sudo touch /etc/puppetlabs/code/environments/production/modules/apache/manifests/init.pp

- Add the class in init.pp :

```
class webserver {
  package { 'nginx': ensure => installed }
  service { 'nginx': ensure => running, enable => true }
}
```

- Include in site.pp
  - File path: /etc/puppetlabs/code/environments/production/manifests/site.pp

```
include webserver
```

- The webserver class ensures Nginx is installed/running across all nodes using the module.

- Scenario
  - A company runs multiple microservices. Each needs Nginx. Instead of repeating the same manifest, they just include the webserver module.

Create the Main Class (init.pp)(ubuntu only)

```
class apache {
# Install Apache package
  package { 'apache2':
    ensure => installed,
  }
  # Ensure service is running
  service { 'apache2':
    ensure => running,
    enable => true,
    require => Package['apache2'],
  }
  # Deploy a custom index.html from module files
  file { '/var/www/html/index.html':
    ensure  => file,
    source  => 'puppet:///modules/apache/index.html',
    require => Package['apache2'],
  } }
```

- Add a Static File (index.html)
  - File path: /etc/puppetlabs/code/environments/production/modules/apache/files/index.html

```
<h1>Welcome to My Apache Server</h1>
```

- Include the Module in site.pp
  - File path: /etc/puppetlabs/code/environments/production/manifests/site.pp

```
node default {
include apache
}
```

Apply the Module on an Agent:

- On the agent node:
  - sudo puppet agent -t
- Puppet contacts the master, compiles the catalog, and applies it:
  - Installs Apache (apache2)
  - Starts the service
  - Copies index.html

Make your module for macOS

```
class apache_macos {
  # macOS ships Apache (httpd) by default
  service { 'org.apache.httpd':
    ensure   => running,
    enable   => true,
    provider => launchd,
  }

  file { '/Library/WebServer/Documents/index.html':
    ensure  => file,
    source  => 'puppet:///modules/apache/index.html',
  }
}
```

Make your module for Windows

```
class apache_windows {
  # Requires Chocolatey to be installed on the Windows agent
  package { 'apache-httpd':
    ensure   => installed,
    provider => chocolatey,
  }
service { 'Apache2.4':
    ensure => running,
    enable => true,
    require => Package['apache-httpd'],
  }
  file { 'C:/Apache24/htdocs/index.html':
    ensure  => file,
    source  => 'puppet:///modules/apache/index.html',
    require => Package['apache-httpd'],
  }
}
```

Assign the right class in site.pp based on node names:

- node 'ubuntu-agent' {
- include apache
- }

- node 'macos-agent' {
- include apache_macos
- }

- node 'windows-agent' {
- include apache_windows
- }

A single file for all three os .

```
class apache {
  case $facts['os']['family'] {
    'Debian': {
      package { 'apache2':
        ensure => installed,
      }
service { 'apache2':
        ensure => running,
        enable => true,
        require => Package['apache2'],
      }
file { '/var/www/html/index.html':
        ensure  => file,
        source  => 'puppet:///modules/apache/index.html',
        require => Package['apache2'],
      }
    }
```

cont ..

- 'Darwin': {
- service { 'org.apache.httpd':
- ensure => running,
- enable => true,
- provider => launchd,
- }

- file { '/Library/WebServer/Documents/index.html':
- ensure => file,
- source => 'puppet:///modules/apache/index.html',
- }
- }

cont..

```
'Windows': {
      package { 'apache-httpd':
        ensure   => installed,
        provider => chocolatey,                    }
      service { 'Apache2.4':
        ensure => running,
        enable => true,
        require => Package['apache-httpd'],              }
      file { 'C:/Apache24/htdocs/index.html':
        ensure  => file,
        source  => 'puppet:///modules/apache/index.html',
        require => Package['apache-httpd'],
      } }
    default: {
      fail("Apache is not supported on ${facts['os']['family']}")
    } }}
```

How this works:

- Puppet will detect the agent OS automatically using facts['os']['family'].
- Depending on the OS family, it will:
- Ubuntu/Debian → install apache2, configure /var/www/html/.
- macOS → enable built-in Apache, configure /Library/WebServer/Documents/.
- Windows → install Apache via Chocolatey, configure C:/Apache24/htdocs/.
- You only need to include apache in your node definition (no separate files).

site.pp

- node default {
- include apache
- }

- Now, when you run the agent:
  - On Ubuntu → it applies the Debian block.
  - On macOS → it applies the Darwin block.
  - On Windows → it applies the Windows block.

- Open a browser on your Mac and go to http://localhost/ — you should see your Puppet-managed index.html.